A Derivation of the Transformer Architecture

Brandon Sandhu

August 31, 2026

Contents

1 Tokens and Embeddings

A *transformer* is just a function $f(x;\theta)$ where x is the input, θ are millions or even billions of learned parameters, which predicts the next token.

1.1 Tokens

Computers do not understand text or language, suppose we have the sentence "The dog sat", the *tokenizer* converts this into vectors. Suppose our vocabulary contains $V = 5$ words, specifically $\{cat, dog, bird, fish, cow\}$, then the vector $e_1 = (1,0,0,0,0,0)^T \in \mathbb{R}^5$ represents cat. Every token has exactly one 1. These are called *one hot vectors*.

These are useful because of the property

$$e_i^T e_i = \begin{cases} 1 & i = j, \\ 0 & i \neq j. \end{cases}$$

This means every word is completely independent of every other word. In linear algebra, we know this to be true as $\{e_i\}$ form the canonical basis of $\mathbb{R}^n$. The limitation here is we have no notion of certain words being *more similar* to other words.

1.2 Embeddings

Therefore, we instead learn an *embedding matrix* $E \in \mathbb{R}^{V\times d}$ where V is the vocabulary size and d is a *embedding dimension*. For example, GPT-3 uses $V = 50257$ and $d = 12288$. The embedding matrix contains one learned vector for every vocabulary token.

Example 1.1. Suppose

$$E = \begin{pmatrix} 2 & 5 \\ 7 & 1 \\ 4 & 3 \end{pmatrix},$$

and our chosen token is the second word in our vocabulary. Its one-hot vector is $v = (1,0,0)^T$, then

$$E^T v = \begin{pmatrix} 2 & 7 & 4 \\ 5 & 1 & 3 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 7 \\ 1 \end{pmatrix}$$

Therefore, multiplication of a one-hot vector by E^T simply selects the correct row embedding representation of our one-hot vector. This can be intuitively seen as a lookup of the embedding vector corresponding to our token in our vocabulary. Implementations of this perform an efficient lookup instead but achieve the same goal. We have shown the following neat result:

Lemma 1.2. *Let $V, d \in \mathbb{N}$ be our vocabulary size and embedding dimension respectively and let $E \in \mathbb{R}^{V\times d}$ be an embedding matrix. For a one-hot vector $e_i \in \mathbb{R}^V$, multiplication by $E^T e_i \in \mathbb{R}^d$ yields our learned vector for the token represented by e_i.*

1.3 Geometry of Embeddings

Every word in our vocabulary becomes a point in the space R^d. So our token *dog* may be represented by a vector v while *wolf* may be represented by a vector w. If learnt *correctly*, these vectors should be *close* together. Therefore, language has become geometry.

To measure *close*, we introduce the notion of a dot product. Given two vectors $v, w \in \mathbb{R}^d$, their dot product is given by

$$v \cdot w = v^T w = \sum_i v_i w_i.$$

Using the identity from linear algebra $v^T w = ||v|| \cdot ||w|| \cos\theta$, we observe that the dot product measures vector length as well as alignment through the angle θ.

1.4 Matrix view of an entire sentence

Suppose our input sentence to a transformer as four tokens. Each embedding has dimension three. Then

$$X = \begin{pmatrix} 0.2 & 1.5 & -0.7 \\ 0.8 & -0.1 & 0.9 \\ -1.2 & 0.5 & 1.4 \\ 0.7 & 0.2 & -0.3 \end{pmatrix},$$

recall rows correspond to our tokens and columns correspond to our features. In general, $X \in \mathbb{R}^{n \times d}$ where n represents the number of tokens in our input and d is our embedding dimension. The matrix X is our true input into the transformer. Here on, we will assume everything operates on X.

The central question of the transformer becomes: Given the matrix X, how can each token gather information form the other tokens in the sequence? To answer this question we must introduct the next major concept of self-attention.

2 Attention

2.1 Why do we require Attention?

Suppose the input sentence is *The animal didn't cross the street because it was tired.*. What does *it* refer to, the street or the animal? Humans instantly recognise it refers to the animal but mathematically our transformer currently has only a matrix

$$X = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}$$

where each row is an embedding. There is no structure which connects these rows. The vector representing *it* has no way to know anything about *animal*. Therefore, we require a mechanism that allows every token to read information from every other token. This is the entire purpose of self-attention.

2.2 What should the output of this process look like

Suppose our input sentence has $n = 4$ tokens represented by x_1, x_2, x_3, x_4 as our embedding vectors. We want a new representation y_i for each token such that y_i depends on all tokens. Therefore, $y_i = f(x_1, \ldots, x_n)$, all that remains is how to define f.

Quite simply, we define

$$y_i = a_{i1}x_1 + a_{i2}x_2 + \cdots + a_{in}x_n$$

where the constants a_{ij} define the importance of token i with each token j. This can be expressed more compactly as the sum

$$y_i = \sum_{j=1}^{n} a_{ij}x_j.$$

Everything else in our transformer exists only to compute the weights a_{ij}. One can visualise the weights a_{ij} as:

Token	**Importance to "it"**
The	0.01
animal	0.72
didn't	0.02
cross	0.01
street	0.05
because	0.04
it	0.10
was	0.02
tired	0.03

Table 1: Token importance scores with respect to the token "it".

Then,

$$y_{\text{it}} = 0.72x_{\text{animal}} + 0.10x_{\text{it}} + \cdots$$

The representation of *it* becomes mostly a copy of *animal*, the exact semantics we wanted to achieve.

Some constraints to be imposed on the weights a_{ij} are: $a_{ij} \geq 0$ as negative importance becomes difficult to interpret and can cause unstability, we also require $\sum_j a_{ij} = 1$ for each token i so that y_i becomes a weighted average, this will prevent vectors from exploding in magnitude.

2.3 Softmax

The softmax function transforms any vector of real numbers into a probability distribution where all values are non-negative and sum to exactly 1. Suppose we have scores $s_1, s_2, \ldots, s_n$. Define

$$a_i = \frac{e_{s_i}}{\sum_k e^{s_k}}.$$

This is the softmax function and immediately one can deduce that $a_i > 0$, as exponentials are always positive and one can show $\sum_i a_i = 1$.

2.4 How to determine the similarity scores given in Table 2

Suppose x_i and x_j are embeddings, the most natural similarity score is given by

$$s_{ij} = x_i^T x_j$$

as a larger score indicates vectors point in similar directions, while a smaller dot product indicates vectors differ and a negative dot product indicates the vectors point in opposite directions. Furthermore, the dot product operator is computationally efficient and differentiable, making it an attractive similarity measure for neural networks.

Suppose we have all our embeddings stacked into $X \in R^{n \times d}$, the similarity between every pair of tokens is given by $S = XX^T \in \mathbb{R}^{n \times n}$, each entry $S_{ij} = x_i^T x_j$ so each row of S tells us how similar token i is to every other token.

Example 2.1. Suppose

$$X = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix}$$

Then,

$$XX^T = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 2 \end{pmatrix}$$

Therefore, as the first row of XX^T is $(1, 0, 1)$ this means token 1 has similarity 1 with itself, 0 with token 2 and 1 with token 3. Each row of XX^T gives us the information of how much attention to pay to every other token.

2.5 A subtle problem

Currently, we have invented a simple attention mechanism which:

- Computes similarity scores: $S = XX^T$
- Applies softmax row-wise: $A = \text{Softmax}(S)$
- Computes weighted averages: $Y = AX$

There is a limitation here, we are using the same embedding vector for three different roles. Prior to the softmax step, we perform the operation XX^T which does $x_i^T x_j$ between embeddings. In this calculation, x_i acts as *what am i looking for* while x_j acts as *what characteristics do I have*, but then when the calculation goes to $x_j^T x_i$, the roles are reversed but the embeddings remain the same. This is where we separate into queries and keys. Then we apply the softmax operator as a normalisation step and in the final calculation of $Y = AX$, we use the same embedding x_i to act as the information being copied but not used for *matching*. Instead of asking the same embedding x_i to play all three roles, we create three different versions. Starting with our input matrix X, we learn three new matrices W_Q, W_K, W_V. Then $Q = XW_Q$, $K = XW_K$ and $V = XW_V$, so that every input token has three corresponding vectors for each role.

This derivation leads direction to the familiar transformer attention equation:

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}V\right).$$

Instead of doing $x_i^T x_j$, we will do $q_i^T k_j$ where this calculation means *what am I searching for*. Then after the attention weights are found through the softmax function, we do $Y = AV$ where V answers *if someone attends to me, what information should I send.*

2.6 Deriving Queries, Keys and Values

We have an input matrix $X \in \mathbb{R}^{n \times d}$ where n is our number of tokens and d is our embedding dimension. Each row represents one token embedding.

Example 2.2. For example, if $n = 5$ and $d_{\text{model}} = 4$ then

$$X = \begin{pmatrix} x_1^T \\ x_2^T \\ x_3^T \\ x_4^T \\ x_5^T \end{pmatrix}$$

notice each row is a vector.

Instead of using X directly, we learn three matrices W_Q, W_K, W_V. These are simply parameter matrices. Suppose $d_{\text{model}} = 512$ and $d_k = 64$, then $W_Q, W_K, W_V \in \mathbb{R}^{512 \times 64}$ are simply projection matrices as discussed before. We require $W_Q, W_K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ to make the calculation compatible however, interestingly W_V is not required to be in the same space. However, the original transformer architecture assumes W_V lies in the same space so we also assume this here. Each of W_Q, W_K, W_V contain learned parameters optimised by gradient descent.

In practice, these matrices W_Q, W_K, W_V are initialised as random and are learnt through backpropagation when the transformer architecture is trained.

To compute queries, Q, we do

$$Q = XW_Q \in \mathbb{R}^{n \times d_k}$$

so that every token has a query vector. Specifically for token i, the query vector is given by $q_i = x_i W_Q \in \mathbb{R}^{1 \times 64}$.

It is worth noting that each query depends only on its own vector, so no communication has occured yet, algebraically this is just a change of coordinate systems.

To compute keys, K, we do the exact same as before:

$$K = XW_K$$

again, $k_i = x_i W_K$, this representation will be learnt to advertise *here is what I contain.*

To compute values, V, we also do

$$V = XW_V$$

and each $v_i = x_i W_V$ is the value vector associated with token i.

For each x_i, we have a corresponding q_i, k_i, v_i, all originating from the same embedding but occupying different learnt subspaces.

The next question is *how much should token i pay attention to token j*, the natural answer is $q_i^T k_j$. This relationship between all tokens is calculated through the matrix product:

$$S = QK^T \in \mathbb{R}^{n \times n},$$

our score matrix.

Example 2.3. Suppose we have three tokens and after projection we obtain:

$$Q = \begin{pmatrix} 1 & 0 \\ 2 & 1 \\ 0 & 2 \end{pmatrix}$$

and

$$K = \begin{pmatrix} 1 & 1 \\ 0 & 2 \\ 2 & 0 \end{pmatrix}$$

then

$$QK^T = \begin{pmatrix} 1 & 0 & 2 \\ 3 & 2 & 4 \\ 2 & 4 & 0 \end{pmatrix}$$

the first row is $(1, 0, 2)$ therefore, the first tokens query has score 1 with token 1, score 0 with token 2 and score 2 with token 3. Before normalisation, the first token *likes* token 3 the most.

However, these scores are just arbitrary real numbers, they are not probabilities therefore we cannot use them as weights in a weighted average of the value vectors. We require a way to transform each row of the scores into a valid probability distribution such that every weight is non-negative, all the weights in a row sum to one and larger scores receive proportionally larger weights.

We already saw the softmax function was a useful tool for this however, before applying softmax the original transformer paper inserts one seemingly mysterious step:

$$S = \frac{QK^T}{\sqrt{d_k}}$$

At first glance, this appears to be an arbitrary scaling factor, however this is false. We will use probability theory to derive why the division by $\sqrt{d_k}$ is necessary. Without it, the dot products grow larger as the dimensionality increases causing the softmax function to become overly peaked and its gradients to vanish.

2.7 Justifying the scaling by $\sqrt{d_k}$

Suppose we ignore the scaling, our attention scores are $S = QK^T$ and each score is $s_{ij} = q_i^T k_j$. Let us examine one score:

Example 2.4. Suppose we have

$$q = \begin{pmatrix} q_1 \\ q_2 \\ \vdots \\ q_{d_k} \end{pmatrix}, \text{ and } k = \begin{pmatrix} k_1 \\ k_2 \\ \vdots \\ k_{d_k} \end{pmatrix},$$

then it follows $q^T k = \sum_{m=1}^{d_k} q_m k_m$. The key question here is *how large should we expect this sum to be*. When training begins, assume the entries of Q and K are approximately centered around zero so that

$$\mathbb{E}[q_m] = 0, \ \mathbb{E}[k_m] = 0$$

and

$$\text{Var}(q_m) = 1, \ \text{Var}(k_m) = 1.$$

Then, consider the product $q_m k_m$, it follows $\mathbb{E}[q_m k_m] = 0$ and therefore, $\mathbb{E}[q^T k] = 0$, so far all is well. However, if the variance is is large then despite having a mean of zero, the values can vary wildly. Next, we compute $\text{Var}(q^T k)$. Since

$$q^T k = \sum_{m=1}^{d_k} q_m k_m,$$

and assuming the terms are independent, it follows

$$\text{Var}\left(\sum_m q_m k_m\right) = \sum_m \text{Var}(q_m k_m),$$

so it suffices to obtain the variance of one product. Recall the formula from probability theory: $\text{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$, since $\mathbb{E}[q_m k_m] = 0$, we have

$$\text{Var}(q_m k_m) = \mathbb{E}[q_m^2 k_m^2]$$

and by assuming independence once again, we have $\mathbb{E}[q_m^2 k_m^2] = \mathbb{E}[q_m^2] \cdot \mathbb{E}[k_m^2]$ but we know $\mathbb{E}[q_m^2] = \text{Var}(q_m) = 1$ and similarly, $\mathbb{E}[k_m^2] = 1$, therefore $\text{Var}(q_m k_m) = 1$, so each term in the dot product contributes variance 1. Therefore it follows

$$\text{Var}(q^T k) = d_k$$

so the variance grows linearly with the embedding dimension. It follows the standard deviation of our score s_{ij} is $\sqrt{d_k}$ so a typical score has magnitude around $\sqrt{d_k}$. To observe why this is bad, we recall the softmax function:

$$\text{Softmax}(s_i) = \frac{e^{s_i}}{\sum_j e^{s_j}}.$$

Exponentials grow very fast, let us compare two cases: for small scores such as $(1, 2, 3)$ exponentials are approximately $(2.7, 7.4, 20.1)$ and softmax gives $(0.09, 0.24, 0.67)$ which seems reasonable. But for larger scores such as $(10, 20, 30)$ exponentials become huge (e^{10}, e^{20}, e^{30}) and softmax yields $(0, 0, 1)$ to numerical precision which such scores gradient descent struggles to adjust the parameters during backpropagation. This is known as softmax saturation.

To fix this issue, we need to keep the variance roughly constant as d_k increases and to do so, we divide our score by the standard deviation

$$\frac{q^T k}{\sqrt{d_k}}$$

, now the variance becomes

$$\text{Var}\left(\frac{q^T k}{\sqrt{d_k}}\right) = 1$$

where we use the property $\text{Var}(cX) = c^2\text{Var}(X)$. This keeps the score in a similar numerical range independent of the dimension d_k which helps keep the softmax operator in a regime where its outputs and gradients are well-behaved.

2.8 The complete attention score

We now have the score

$$S = \frac{QK^T}{\sqrt{d_k}} \in \mathbb{R}n \times n$$

where each row contains the scaled compatibility scores between one token's query and every tokens key. Once we have applied the softmax operator row by row to the score and apply this to our values, we obtain the famous attention equation given by

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V.$$

3 Multi-Head Attention

So far we have built a complete single-head attention mechanism however modern transformers use 8, 16, 32 attention heads instead of a single head. This is because multiple heads allow the model to learn different kinds of relationships simultaneously. One head may specialise in grammatical dependencies while another in semantic similarity.

Take the following sentence as example:

"The scientist who won the Nobel Prize thanked her mentor because she inspired her."

Focus on the word *she*, to understand this sentence, different relationships are useful:

- Which noun does *she* refer to
- What is the main verb

- Where does the subordinate clause begin
- Which words describe the scientist

These are different kinds of relationships, a single attention distribution has to represent all of them at once. Recall a single attention head computes $A = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)$, therefore for each token there is exactly one probability distribution. Suppose token i wants to be

- strongly attend to the subject
- moderately attend to the verb
- weakly attend to punctuation

those priorities are all mixed into one distribution. The solution taken here is instead of learning one set of projections $\{W_Q, W_K, W_V\}$, we learn several.
Suppose we have $h = 8$ heads, then for head r, we learn

$$W_Q^{(r)},\ W_K^{(r)},\ W_V^{(r)}$$

so each head has its own parameters. Therefore, for head $r = 1$, we have

$$Q^{(1)} = XW_Q^{(1)},\ K^{(1)} = XW_K^{(1)},\ V^{(1)} = XW_V^{(1)}$$

then

$$H^{(1)} = \text{Softmax}\left(\frac{Q^{(1)}K^{(1)T}}{\sqrt{d_k}}\right)V^{(1)}.$$

Importantly note that each head computes attention independent of the others, there is no communication between them. For dimensional bookkeeping if we have $d_{\text{model}} = 512$ then each head uses $d_k = 512/8 = 64$ so that every head produces $H^{(r)} \in \mathbb{R}^{512\times 64}$ so that when we concatenate each head, we recover 512 dimensions. Intuitively, we are dividing the representational space among multiple specialists rather than increasing the total dimensionality.

3.1 Concatenation

Suppose we have $H^{(1)}, H^{(2)],\ldots,H^{(8)}}$ heads each sitting in $\mathbb{R}^{n\times 64}$. We concatenate along the features dimension, so

$$H = \text{Concat}(H^{(1)}, H^{(2)}, \ldots, H^{(8)})$$

so that $H \in \mathbb{R}^{n\times 512}$. If we were to add these matrices, we would be combining the information from each head, so concatenation as defined here preserves the information from each head.

3.2 The Final Projection

After concatenation, we have one final step. We multiply by one more learned matrix $W_0 \in \mathbb{R}^{512\times 512}$, so that the output becomes $Y = HW_0$, or written in a single expression:

$$\text{MultiHead}(X) = Concat(H^{(1)}, H^{(2)}, \ldots, H^{(8)})W_0,$$

where each

$$H^{(r)} = \text{Attention}(Q^{(r)}, K^{(r)}, V^{(r)}).$$

Currently, with just concatenation, the output vector for a token looks something like:

$$\begin{pmatrix} \text{Head 1 features} \\ \text{Head 2 features} \\ \vdots \\ \text{Head 8 features} \end{pmatrix}$$

at this stage the heads are stacked together but have not interacted with each other. the output projection W_0 is a learned linear transformation that allows the model to mix information across heads, without this the next layer would receive eight independent blocks of features rather than one integrated representation.

4 Residual Connections

The next question to consider is *if attention already produces a better representation, why don't we simply replace the old representation with a new one*

Instead, transformers do something which initially looks wasteful:

$$X \mapsto X + \text{Attention}(X).$$

This map is called a *residual connection*. We require these because suppose one transformer layer computes $F(X) = \text{MultiHead}(X)$, without a residual connection, the output would simply be $Y = F(X)$ and each layer would replace the previous representation. The problem with this is described below.

Suppose we are stacking 48 transformer layers, and suppose layer 1 learns something useful, layer 2 slightly improves it, layer 3 accidentally makes it worse. Then layer 4 has to undo the mistake made in layer 3. The deeper the network becomes, the harder it is to preserve the useful information. Essentially, each layer is forced to learn both: new information and how not to destroy old information. If we have trained a 20-layer network, and we add a new layer, if this new layer is not useful then it should simply have no effect on the network i.e. we would like the effect of adding a new layer to behave as $Y = X$. Neural networks struggle to learn the identity function. Generally, a layer computes something like $F(X) = \sigma(XW + b)$ where W is some weights, b is some drift and σ is a nonlinearity. Part of the reason they struggle to learn the identity function is because of non-linear activation functions.

Therefore, instead of asking the layer to learn $Y = F(X)$, we ask it to learn $Y = X + F(X)$, if the best thing a layer can do is learn nothing then it will just learn $F(X) = 0$ which is easy.

Example 4.1 (Taylor series analogy)**.** Suppose the ideal transformation is only a small modification of the input i.e. $Y = X + \epsilon(X)$ where $\epsilon(X)$ is relatively small, this resembles the first terms of a Taylor expansion. This is intuitively similar to the residual connections we describe above. Many useful transformations in deep learning are incremental refinements rather than

complete rewrites.

4.1 The gradient problem with identity approach

Consider N layers, without residual connections they look like:

$$X \to F_1 \to \cdots \to F_N \to L$$

where L is our loss function. During backpropagation the chain rule gives us

$$\frac{\partial L}{\partial X} = \prod_{i=1}^{N} \frac{\partial F_i}{\partial F_{i-1}}.$$

The key is that this is a product of many matrices, so if these intermediate gradients are *smaller than one* then multiplying many of them essentially makes the gradient disappear. This is known as the *vanishing gradient problem*. If the early layers receive almost zero gradient, they learn extremely slowly.

With residual connections, suppose $Y = X + F(X)$. Then it follows

$$\frac{\partial Y}{\partial X} = I + \frac{\partial F}{\partial X},$$

where I is the identity matrix. Now even if the contribution of $\frac{\partial F}{\partial X}$ becomes small, the gradient still has the contribution from I. So instead of having a zero gradient, we have a gradient passing through unchanged which is a main reason why deep transformers are able to be trained successfully. With the residual connection where there is minimal change in the second term, the gradient is able to *skip* through the layers during back propagation, this is why residual connections are also referred to as *skip connections*.

4.2 Layer Normalisation

The attention output can have varying scales from one batch to the next. If these scales drift too much, optimisation becomes harder. Transformers often apply a *Layer Normalisation* step after the residual connections to stabilise the representations. A simplified transformer block looks like this:

$$X \to \text{MultiHeadAttention} \to X + \text{Attention}(X) \to \text{LayerNorm}.$$

Then the process repeats with the feed-forward network

$$\text{LayerNorm} \to \text{MLP} \to \text{Residual} \to \text{LayerNorm}.$$

Example 4.2. Historically, many modern large language models, including GPT-style models, use a pre-normalisation (Pre-LN) architecture, where LayerNorm is applied before the attention and MLP sublayers rather than after the residual addition. The mathematical purpose is the same, for stable optimisation, but the placement changes the gradient flow and tends to make training very deep models easier.

5 The Feed-Forward Network (MLP)

Many people think that the attention mechanism is the transformer, however this is false. In modern transformers, roughly two-thirds of the parameters often live in the MLP, not in attention.

5.1 MLP (Multi-Layer Perceptron

Despite the fancy name, it is just a small neural network. The simplest MLP looks like:

$$x \to Wx + b \to \sigma \to Ux + c$$

so, a linear transformation step, a non-linear activation, and another linear transformation step. One may question why one linear transformation isn't sufficient, suppose we have $y = Wx$, then add another layer $z = Uy$, then we have $z = U(Wx) = (UW)x$, so the two linear layers collapse into one. Therefore, adding more linear layers does not increase the expressive power. Instead the second step is to compute using an activation function, $y = \sigma(Wx)$ and then do $z = U\sigma(Wx)$, this can no longer be simplified into a single matrix because of the non-linear function. The activation is what gives neural networks their expressive power.

Originally, transformers use the ReLU activation, defined as

$$\text{ReLU}(x) = \max(0, x).$$

Modern large language models more commonly use smoother activations such as GELU or SiLU, for example

$$GELU(x) = x\Phi(x),$$

where $\Phi(x)$ is the cumulative distribution function of the standard normal distribution. The key part here is that these functions are non-linear for the reason explained before.

5.2 The Transformers MLP

Suppose $d_{\text{model}} = 512$, the MLP first expands dimension from 512 to say 2048 through

$$H = XW_1 + b_1$$

where $W_1 \in \mathbb{R}^{512\times2048}$. Then apply GELU $H' = GELU(H)$, and finally project back to 512 dimension embedding vectors through

$$Y = H'W_2 + b_2$$

where $W_2 \in \mathbb{R}^{2048\times512}$. The reason for expanding the dimension instead of staying in $d_{\text{model}} = 512$ dimensions can be explained through the following geometrical analogy: Suppose you are trying to separate two intertwined curves. In two dimensions, they may be impossible to separate with a straight line. Lift them into three dimensions and a plane can now separate them. The MLP does a similar thing, by expanding into a higher-dimensional feature space, the network can construct and manipulate intermediate features before compressing them back to the

model dimension.

But one may argue that shouldn't attention ahve already produced a good representation, why would we require another neural network. What attention really does is the following: recall $Y = AV$, for one token $y_i = \sum_j a_{ij} v_j$, observe that every output is a weighted average, these are linear operations. Attention is able to where to look and how much to copy but it cannot, by itself, invent highly nonlinear transformations of the information it has gathered.

Example 5.1. Suppose attention gathers information about a person: [language=json] "occupation": "scientist", "country": "uk", "won$_p rize$" : $true$,"age" : 45 attention has assembled the relevant facts but now suppoer we want to infer that *this person is probably academically influential*, this conclusion is not obtained through copying information from another token; it requires combining features and applying a learned non-linear transformation. The MLP performs this transformation.

Example 5.2. Attention answers the question of *who should I listen to* while the MLP answers *now that I've heard everyone, how should I think about what I learned*, these are two fundamentally different operations, one is linear in nature and the other is non-linear.

Suppose $X \in \mathbb{R}^{n \times 512}$ is our input. We know the rows are tokens and the columns are features. Attention computes the matrix product, AX, where $A \in \mathbb{R}^{n \times n}$, therefore this product mixes rows. Each output token becomes a combination of multiple input tokens. The calculation does not mix the features within a token; every value vector is treated as a whole during the weighted averaging.

The MLP works differently, for each token independently,

$$x_i \to \text{MLP}(x_i),$$

so it must be noted that the MLP layer is applied to each token independently, every row in the output of attention passes through the exact same MLP network, no communication between tokens occurs here. The MLP is mixing the columns (features) within each row.

- Attention mixes information across tokens
- The MLP mixes information across features within each token

One may ask shouldn't each token have its own neural network, suppose the input sentence has 100 tokens, this would mean we require 100 different MLPs, instead transformers use one shared MLP:

$$f(x) = W_2 \sigma(W_1 x + b_1) + b_2,$$

because it makes the process more efficient using one MLP as there are far fewer parameters to leearn and the model learns transformations that work regardless of a token's position in the sequence.

5.3 A complete transformer block

One transformer block can be written schematically as

$$X_1 = X + \text{Attention}(X), X_2 = \text{LayerNorm}(X_1) X_3 = X_2 + MLP(X_2) Y = \text{LayerNorm}(X_3$$

This repeated alternation of communicate, compute, communicate compute is what allows transformers to build increasingly rich representations over many layers.

6 Training

We have built the architecture of a transformer block, but so far it is a complicated function with random initialised weights. We aim to answer the question of *how does such a model learn English*, everything discussed thus far - queries, keys, values, attention heads, MLPs - is initially random noise.

Suppose we have passed our input sentence through 24 transformer blocks. The final output looks like $H \in \mathbb{R}^{n \times d_{\text{model}}}$. For GPT-style models, $d_{\text{model}} = 4096$ or even larger. At this point, each token has a rich contextual representation. Suppose we are predicting the next word after *The cat sat on the*. The final hidden vector for the last token is $h \in \mathbb{R}^{4096}$, the last row of our matrix H. Currently, this is a vector which is the model's internal understanding of everything it has read so far, we need to somehow convert it into probabilities over the vocabulary.

6.1 The Vocabulary Projection

Suppose the vocabulary contains $V = 50000$ tokens. We introduce one final matrix

$$W_{\text{vocab}} \in \mathbb{R}^{4096 \times 50000},$$

and perform the following multiplication

$$z = hW_{\text{vocab}} \in \mathbb{R}^{1 \times 50000}$$

We now have a vector z with one real number for each vocabulary word. These numbers are known as *logits*. Suppose $z = (4.2, 1.1, -3.4, 0.8, \dots)$, these are not probabilities they are simply scores, the larger the logit for a specific vocabulary word, the more the model favors that word. Negative values are perfectly acceptible. Only relative values matter here.

The matrix multiplication of $z = hW_{\text{vocab}}$ is simply a collection of dot products, suppose column 137 of W_{vocab} corresponds to the word *cat*, then $z_{137} = h^T w_{\text{cat}}$ is just a dot product describing some similarity score of *how similar is my current understanding of the context to the learned representation for the word cat.*

A trick of *weight tying* is often used by modern transformers where instead of learning a completely separate output matrix W_{vocab}, they use $W_{\text{vocab}} = E^T$. Recall $E \in \mathbb{R}^{V \times d_{\text{model}}}$. This works because the same learned geometry that maps words into vectors on the way in can also map vectors back to words on the way out. This reduces the number of parameters and often improves performance.

6.2 Softmax again

The next step in obtaining our probabilities of the next word in the input sentence is through the softmax function again. Suppose our logits are:

Word	Logit
cat	5.1
dog	3.8
house	-1.4
banana	-4.2

Table 2: Logits for different vocabulary words

We convert them into probabilities using the softmax function

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}},$$

here instead of normalising across tokens as we did in attention, we are normalising across the vocabulary.

Example 6.1. Suppose $z = (2, 1, 0)$. The exponentials are $(7.39, 2.72, 1)$ and the sum of these are 11.11. Therefore, the probabilities are $(0.665, 0.245, 0.090)$. The model predicts the first word with probability 66.5%.

Suppose the next true word in our sentence of *The cat sat on the* is *mat*, ideally we would like the probability of *mat* being 1.0 and everything else being 0.0. However, the model of course never achieves exactly that, instead training gradually pushes the probability mass toward the correct word.

6.3 How do we measure wrong

Most people will think that using mean squared error (MSE) is a suitable loss function because it is common in regression, suppose the true distribution is $y = (0, 1, 0)$ and the model predicts $p = (0.2, 0.6, 0.2)$. Using MSE gives us

$$(0.2)^2 + (0.4)^2 + (0.2)^2$$

the reason we don't use MSE is because the model is not trying to predict a number like regression does, it is trying to predict a probability distribution. We want to answer the question *how surprised should the model be by the actual next token*, the concept of *surprise* has a natural mathematical measure.

Suppose an event has probability p. It was shown by Claude Shannon that the information content (or surprise) of observing that event is $-\log p$. This is exactly the behaviour we want, if $p = 1$ then $-\log(1) = 0$ so no surprise, if $p = 0.5$ then $-\log(0.5) \approx 0.69$ moderately surprising. Finally, if $p = 0.001$ then $-\log(0.001) \approx 6.91$, extremely surprising. This penalty is exactly what we want to penalise during training.

6.4 Cross Entropy

Suppose the true distribution is y, so it is 1 where the word *mat* sits and 0 else where. This is known as a one-hot distribution. Cross entropy measures *how much probability did the model assign to the correct answer* the loss function is given by

$$L = -\sum_i y_i \log p_i.$$

Since the true distribution y is one-hot, only one term survives in this loss function, $-\log p(\text{mat})$. So the surprise is returned as the loss function. A larger surprise means a greater loss.

Example 6.2. Suppose our training data consists of token sequences sampled from some true but unknown distribution.
Maximising the probability that our model assigns to the observed next token is known as *maximum likelihood estimation.*
Taking the logarithm turns a product of probabilities into a sum of log-probabilities, making optimisation practical. Negating this yields negative log-likelihood, but when the target is one-hot, this yields exactly the cross entropy loss function above.
Therefore, transformers are not minimising an arbitrary loss function, they are performing maximum likelihood estimation under a probabilistic model of language.

7 Backpropagation

We have understood how the model produces probabilities for the next probable word, why softmax is used during this process and why cross entropy is the correct choice of loss function. The next natural question is *how does the model know how to change billions of parameters after seeing one incorrect prediction.* We have multiple weights such as the embedding matrix, every attention head, every query, key and value matrix, every MLP weight, every output projection. We make use of backpropagation to update all these at once.
Everything built thus far is just a complicated function $L(\theta)$ where θ is every parameter in our transformer and L is the cross-entropy loss function. Training means solving the following optimisation problem:

$$\min_\theta L(\theta).$$

7.1 Backpropagation from First Principles

Let us focus on a much simplier problem, forgetting transformers for a moment. Suppose $y = 3x$ and $L = y^2$. This process can be represented as a computational graph:

$$x \to \times 3 \to y \to \text{square} \to \text{Loss}$$

So with $x = 2$, $y = 6$ and $L = 36$. The question is *if I change x slightly, how much does the loss change*, this is exactly what the derivative $\dfrac{dL}{dx}$ describes. One could naively substitute $L = (3x)^2$ and differentiate directly, however neural networks are far too complicated for this. Instead, one uses the chain rule:

$$\frac{dL}{dx} = \frac{dL}{dy}\frac{dy}{dx}$$

so with $L = y^2$, we have $\frac{dL}{dy} = 2y$ and with $y = 6$, this becomes 12. Next, with $y = 3x$, we have $\frac{dy}{dx} = 3$ so $12 \times 3 = 36$. Therefore, $\frac{dL}{dx} = 36$. The derivative tells us exactly which direction to move when we change x slightly.

With $x = 2$ and the gradient being 36, with a fixed learning rate of $\mu = 0.1$, then gradient descent tells us to perform the following update to x

$$x \leftarrow x - 0.1(36) = -1.6$$

because we are moving in the direction that decreases the loss. Now replace one parameter with a matrix of parameters so suppose we have W_Q, again we compute $\frac{\partial L}{\partial W_Q}$ using the chain rule. This is a matrix of same dimensionality as W_Q containing gradients for each parameter. Gradient descent updates each element simultaneously.

Suppose $Y = XW$, this fundamental equation appears everywhere from embeddings, queries, keys, values, MLPs, output projections. Suppose $L = L(Y)$, we know what $\frac{\partial L}{\partial Y}$ is but we want $\frac{\partial L}{\partial W}$. The result for this is

$$\frac{\partial L}{\partial W} = X^T \frac{\partial L}{\partial Y}.$$

To show this is simple. Suppose

$$X = \begin{pmatrix} x_1 & x_2 \end{pmatrix} \text{ and } W = \begin{pmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{pmatrix}$$

with $Y = XW$, then expanding we have

$$y_1 = x_1 w_{11} + x_2 w_{21} \tag{1}$$
$$y_2 = x_1 w_{21} + x_2 w_{22} \tag{2}$$

Suppose we want the quantity $\frac{\partial L}{\partial w_{11}}$, by the chain rule we know

$$\frac{\partial L}{\partial w_{11}} = \frac{\partial L}{\partial y_1} \frac{\partial y_1}{\partial w_{11}}$$

and $\frac{\partial y_1}{\partial w_{11}} = x_1$, so

$$\frac{\partial L}{\partial w_{11}} = x_1 \frac{\partial L}{\partial y_1}.$$

Repeating this for each element of W, the collecgtion of derivatives forms exactly

$$X^T \frac{\partial L}{\partial Y}.$$

Recall we have $Q = XW_Q$, during backpropagation suppose we know $\frac{\partial L}{\partial Q}$, then immediately

$$\frac{\partial L}{\partial W_Q} = X^T \frac{\partial L}{\partial Q}$$

and the same applies for W_K, W_V, W_O, the MLP matrices and the vocabulary projection.

Observe that the weights are not the only input into a new layer, X is also an input into a new layer so it also requires updating. Therefore, we also require $\frac{\partial L}{\partial X}$, with $Y = XW$, the result is $\frac{\partial L}{\partial X} = \frac{\partial L}{\partial Y} W^T$.

7.2 Applying Backpropagation to Attention

Recall our forward computation of $Q = XW_Q, K = XW_K$ and $V = XW_V$. Suppose our loss tells us that the attention output should have been different, backpropagation computes gradients flowing back through the attention mechanism until it reaches Q, K and V. Then using our formulas from before, we have

$$\frac{\partial L}{\partial W_Q} = X^T \frac{\partial L}{\partial Q} \tag{3}$$

$$\frac{\partial L}{\partial W_K} = X^T \frac{\partial L}{\partial K} \tag{4}$$

$$\frac{\partial L}{\partial W_V} = X^T \frac{\partial L}{\partial V} \tag{5}$$

and

$$\frac{\partial L}{\partial X} = \frac{\partial L}{\partial Q} W_Q^T + \frac{\partial L}{\partial K} W_K^T + \frac{\partial L}{\partial V} W_V^T$$

which follows from the construction of Q, K and V from X. It is worth noting that all formulae up until now are assuming we know the quantity $\frac{\partial L}{\partial Q}$ and same for K and V, we will compute these later.

Currently, we have derived how gradients pass through the linear layers, however there are still two non-linear operations inside attention: the matrix multiplication QK^T and the row-wise softmax operator. The softmax derivative is particularly elegant as its Jacobian has the form

$$\frac{\partial s_i}{\partial z_j} = s_i(\delta_{ij} - s_j)$$

where δ_{ij} is the Kronecker delta so $\delta_{ij} = 1$ if and only if $i = j$, 0 otherwise. The key point here is for non-linear functions we will be using its Jacobian to approximate the derivative.

7.3 A walkthrough backpropagation across a single transformer layer

We know our loss function is $L = -\sum_i y_i \log p_i$ where y is our one-hot true output given by

$$y_i = \begin{cases} 1 & \text{if } i \text{ is the correct word} \\ 0 & \text{otherwise} \end{cases}$$

and p_i is the predicted probability of the vocabulary word i. Therefore, the loss function simplifies to

$$L = -\log p_k$$

where p_k is the probability of the correct vocabulary word k. Since we know $p_k = \dfrac{e^{z_k}}{\sum_j e^{z_j}}$, our loss function simplifies to

$$L = -z_k + \log \sum_j e^{z_j}$$

Therefore, it follows that

$$\frac{\partial L}{\partial z} = p - y$$

This formula tells us that by changing our logits slightly, it impacts our loss function by a change of $p - y$, so if $\dfrac{\partial L}{\partial z_{\text{Cat}}} < 0$ during backpropagation, then this tells us to *increase the cat logit* to reduce our loss function. We know these logits are computed from $z = HW_{\text{vocab}}$, where $H \in \mathbb{R}^{n \times d_{\text{model}}}$ is the final hidden representation. From the chain rule, we know

$$\frac{\partial L}{\partial H} = \frac{\partial L}{\partial z} W_{\text{vocab}}^T$$

so now we know the impact on the loss function when we change the final hidden layer slightly. This is repeated across the whole computation graph and we can realise how changing parameters slightly impacts the loss function and updates the parameters accordingly.

7.4 Updating parameters through gradient descent on a tiny linear network

In this section we will perform one complete gradient update with a tiny network example. All other linear layers in our transformer behave exactly the same way. Suppose our network is just $y = xW$ where

$$x = \begin{pmatrix} 2 & 1 \end{pmatrix} \text{ and } W = \begin{pmatrix} 1 & 3 \\ 2 & 4 \end{pmatrix}$$

We perform our forward pass, calculating using the numbers above we obtain

$$y = xW = \begin{pmatrix} 4 & 10 \end{pmatrix}.$$

Suppose our desired output is $t = \begin{pmatrix} 5 & 8 \end{pmatrix}$, we will use a mean squared error loss function for simplicity here, $L = \frac{1}{2}\sum_i (y_i - t_i)^2$, so with the results above our loss is 2.5. Next, we compute our gradients

$$\frac{\partial L}{\partial y} = y - t$$

so $\dfrac{\partial L}{\partial y} = \begin{pmatrix} -1 & 2 \end{pmatrix}$. This tells us that we need to increase the first output and decrease the second output. Now we want to compute the gradients with respect to the weights, W. We know

$$\frac{\partial L}{\partial W} = x^T \frac{\partial L}{\partial y}$$

and we compute that $x^T = \begin{pmatrix} 2 \\ 1 \end{pmatrix}$ so we obtain that

$$\frac{\partial L}{\partial W} = \begin{pmatrix} 2 \\ 1 \end{pmatrix} \begin{pmatrix} -1 & 2 \end{pmatrix} = \begin{pmatrix} -2 & 4 \\ -1 & 2 \end{pmatrix}$$

so every weight has its own derivative for example $\dfrac{\partial L}{\partial W_{12}} = 4$, this means if we increase our weight W_{12} slightly, the loss will increase. So we should reduce this weight. The update rule is given by

$$W_{\text{new}} = W - \mu \frac{\partial L}{\partial W}$$

where μ is a learning rate. Computing this with $\mu = 0.1$, we have

$$0.1 \begin{pmatrix} -2 & 4 \\ -1 & 2 \end{pmatrix} = \begin{pmatrix} -0.2 & 0.4 \\ -0.1 & 0.2 \end{pmatrix}$$

so subtracting this from our original weights, yields the new weights of

$$W_{\text{new}} = \begin{pmatrix} 1 & 3 \\ 2 & 4 \end{pmatrix} - \begin{pmatrix} -0.2 & 0.4 \\ -0.1 & 0.2 \end{pmatrix} = \begin{pmatrix} 1.2 & 2.6 \\ 2.1 & 3.8 \end{pmatrix}$$

So our weights have moved in directions that should reduce the loss. We then run another forward pass using the updated weights. We yield $y_{\text{new}} = \begin{pmatrix} 4.5 & 9 \end{pmatrix}$ and a loss of 0.625. Observe our new loss is 75% less than the initial loss. Repeating like this is how the weights get optimised.

8 Conclusion

To conclude, we have shown how to build a transformer architecture, beginning with tokenisation and embeddings, which convert sentences into vector representations. We then introduced the attention mechanism, explaining how it enables the model to learn relationships between tokens by allowing each token to interact with every other token. Building on this, we discussed multi-head attention, which allows the model to capture different types of patterns and relationships simultaneously.

Next, we examined residual connections and their role in improving the training process by facilitating gradient flow through deep networks. We also highlighted the importance of the multilayer perceptron (MLP) layer, which introduces non-linearity and enables richer interactions within the feature space of each individual token.

Finally, we described the training process, showing how backpropagation updates the model's parameters by minimising a carefully designed loss function. We motivated this loss function using the information-theoretic concept of surprise, which provides a principled measure of how well the model's predictions align with the observed data.